



Facultad Regional Tucumán

Departamento de Electrónica

Técnicas Digitales II

Actividad de Formación Práctica 4

Tema: Programación de microcontroladores: Funciones no bloqueantes, aplicación con SysTick en STM32CubeIDE.

Grupo 6

Profesor:

Ing. Rubén Darío Mansilla

ATTP:

Ing. Lucas Abdala

Grupo 6:

Tejerina Damián, Leal Franco, Jiménez Emanuel, Porcel Guillermo

Introducción:

Muchas veces se necesita esperar un tiempo antes de ejecutar algo: leer un sensor, cambiar el estado de un LED, enviar un dato, etc. Para lograrlo existen funciones de retardo. Sin embargo, no todas funcionan igual, por un lado tenemos las bloqueantes: son funciones que detienen completamente la ejecución del programa durante un tiempo determinado, también tenemos las no bloqueantes: son funciones donde se espera un tiempo sin detener el programa. Se basan en contadores, timers o marcas de tiempo, pero el micro sigue ejecutando código mientras tanto.

¿Qué hacen?

- Permiten ejecutar muchas tareas en paralelo dentro del mismo loop.
- No bloquean la CPU.
- Permiten leer sensores, atender interrupciones, controlar comunicaciones, etc. mientras "se espera".

Ventajas

- Mucho mejor rendimiento: la CPU aprovecha cada ciclo.
- Estabilidad: se pueden manejar múltiples temporizaciones a la vez.
- Experiencia fluida: el sistema responde rápido a eventos.
- Evitan lentitud causados por delays grandes.
- Son compatibles con sistemas cooperativos, multitarea simple y máquinas de estados.

Creación de la librería API_Delay

Header

En el encabezado de la librería seguimos los siguientes pasos:

- 1- Incluimos las librerías `<stdint.h>` y `<stdbool.h>`.
- 2- Creamos el typedef `"tick_t"` (enteros de 32 bits que guardan el tiempo transcurrido en milisegundos)
- 3- Creamos el typedef `"bool_t"` (variables booleanas, que guardan true o false).
- 4- Creamos la estructura `delay_t`
- 5- Declaramos las funciones: `delayInit`, `delayRead`, `delayWrite`.

```

2  * API_Delay.h
3  *
4  *   Created on: Oct 12, 2025
5  *   Author: Zenhyc
6  */
7
8  #include <stdbool.h>
9  #include <stdint.h>
10
11
12  #ifndef API_INC_API_DELAY_H_
13  #define API_INC_API_DELAY_H_
14
15  typedef uint32_t tick_t;
16  typedef bool bool_t;
17
18  typedef struct{
19      tick_t startTime;
20      tick_t duration;
21      bool_t running;
22  }delay_t;
23
24  void delayInit(delay_t *delay, tick_t duration);
25  bool_t delayRead(delay_t *delay);
26  void delayWrite(delay_t *delay, tick_t duration);
27
28
29  #endif /* API_INC_API_DELAY_H_ */
30

```

Source

En el archivo de extensión “.c”, implementamos las funciones que declaramos en la cabecera.

delayInit

En esta función inicializamos el delay, recibimos de parámetro un tiempo de duración el cual le asignamos mediante el atributo *duration* de la estructura *delay_t*. Luego iniciamos *running* en *false*, esto se hace porque el inicio del conteo se hace al llamar a otra función llamada “*delayRead*”.

delayRead

Esta función revisa si ya paso el tiempo correspondiente al delay previamente definido, esto lo hace a través del SysTick, con la función “*HAL_GetTick()*” que devuelve el tiempo transcurrido en milisegundos desde que inicio la aplicación en el microcontrolador.

Si *running* es *false*, lo que hace es iniciar el conteo, almacenando en la variable *starTime* el tiempo en el que comienza el conteo, luego establece *running* en *true*, ya que el contador está corriendo. Si *running* es *true*, esto significa que el delay está en ejecución por lo tanto a cada llamado de la función corrobora si el tiempo de duración se ha cumplido, de ser así retorna *true* a la función informando que el delay ha finalizado y establece *running* en *false* ya que el delay dejo de contar. Si el tiempo de duración todavía no se cumplió retorna *false* a la función.

delayWrite

Lo único que hace esta función es permitir sobrescribir el tiempo de duración del delay.

```

2  * API_Delay.c
3  *
4  * Created on: Oct 12, 2025
5  * Author: Zenhvr
6  */
7
8  #include "main.h"
9  #include "API_Delay.h"
10
11 void delayInit(delay_t *delay, tick_t duration){
12     delay->startTime=HAL_GetTick();
13     delay->duration=duration;          //valor que tomara al definir el tiempo de espera
14     delay->running=false;
15 }
16 bool_t delayRead(delay_t *delay){
17     if(delay->running==false){
18         delay->startTime=HAL_GetTick();
19         delay->running=true;
20         return false;
21     }
22     if((HAL_GetTick()-delay->startTime)>=delay->duration){
23         delay->running=false;
24         return true;
25     }
26     return false;
27 }
28 void delayWrite(delay_t *delay, tick_t duration){          //Permite cambiar el tiempo de duracion de un delay existente
29     delay->duration=duration;
30 }
31

```

ENLACES:

REPOSITORIO GRUPAL:

[Grupo 6 TDII 2025/AFP 4 GRUPO 6 TDII 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

APP 1 1: [Grupo 6 TDII 2025/AFP 4 GRUPO 6 TDII 2025/App 1 1 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

APP 1 2: [Grupo 6 TDII 2025/AFP 4 GRUPO 6 TDII 2025/App 1 2 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

APP 1 3: [Grupo 6 TDII 2025/AFP 4 GRUPO 6 TDII 2025/App 1 3 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)

APP 1 4: [Grupo 6 TDII 2025/AFP 4 GRUPO 6 TDII 2025/App 1 4 GRUPO 6 2025 at master · ZePhyR1906/Grupo 6 TDII 2025](#)